

NodeJS

Présentation

Architecture

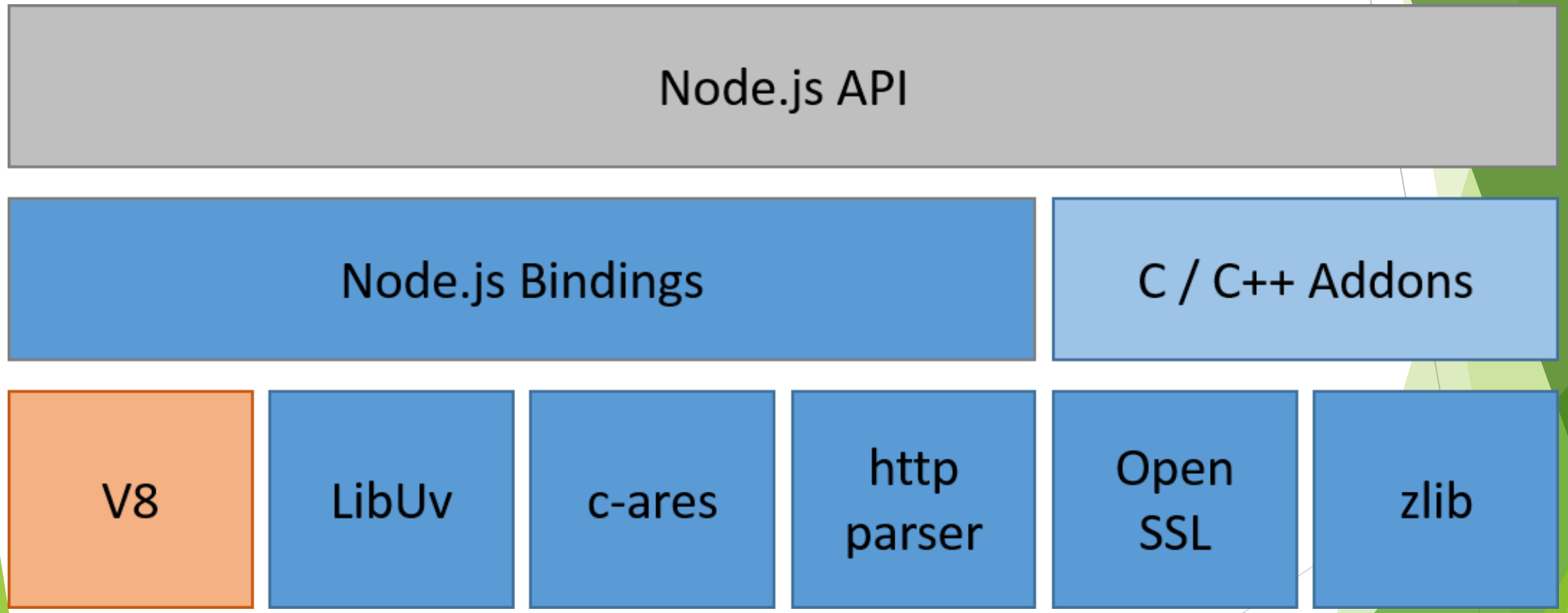
The reactor patterns

Modules

NodeJS

- Node.js est un environnement d'exécution open source et multiplateforme permettant de créer des applications côté serveur et réseau à l'aide de JavaScript
- Basé sur le moteur JavaScript V8 de Chrome
- Fournit une API d'E/S non bloquante pilotée par des événements
- Le projet a fait l'objet d'une démonstration lors de la première édition européenne de la JSConf le 8 novembre 2009.

Architecture



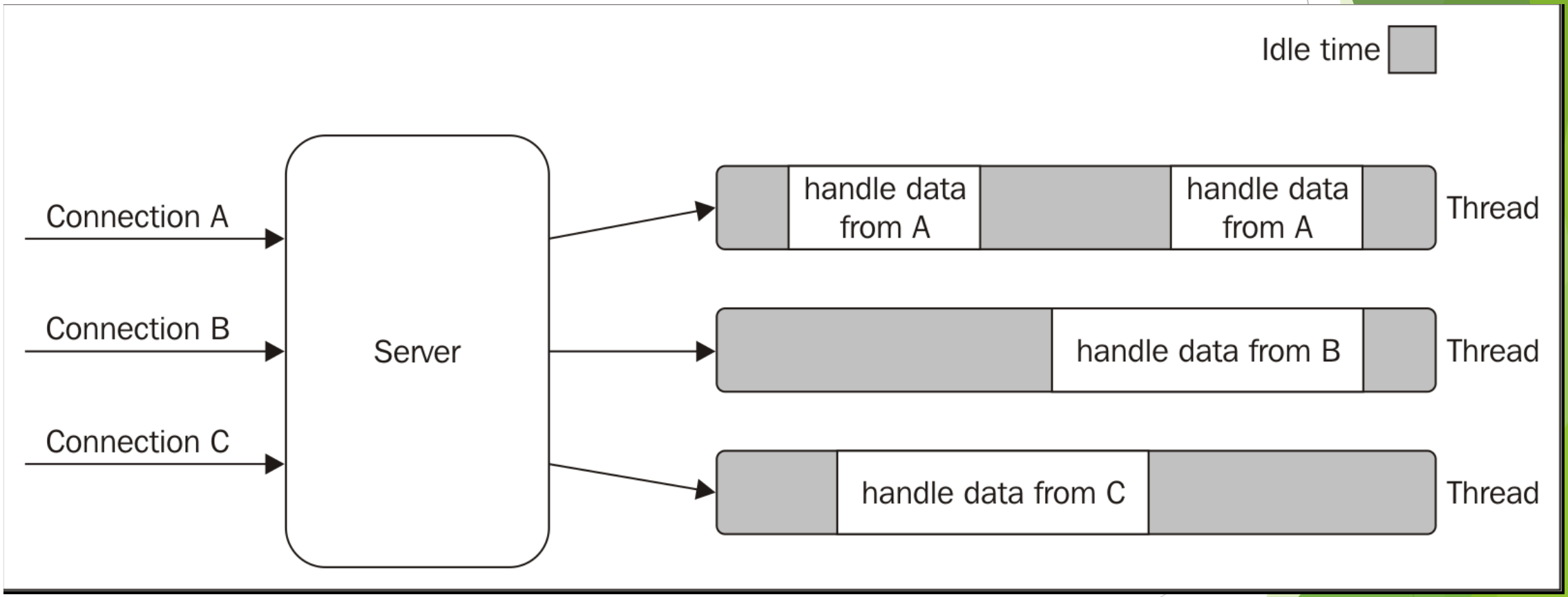
The reactor pattern

- Design Pattern pour gérer efficacement les appels I/O
  - ***"event loop"*** : boucle qui écoute les événements et appelle le handler correspondant lorsqu'un événement est produit
  - ***"event handler"*** : callback ou promise pour gérer les événements qui se produisent
- Représente le cœur de la nature asynchrone de Node.js
- se base sur
  - architecture single-threaded
  - non blocking I/O

I/O

- I/O est la plus lente opération dans un ordinateur
  - RAM: 10^{E}-9 seconds; GB/s
  - disque: 10^{E}-3 seconds; MB/s
- I/O n'est pas couteux en terme de processeur mais elle rajoute un délai entre les opérations
- I/O est bloquant dans les langages traditionnels:
  - un appel à une opération I/O bloque l'exécution jusqu'à la fin de l'opération
- Mécanismes pour gérer le blocage des opérations I/O:
  - Multi-threading
  - Busy-waiting
  - Event loop

blocking I/O


Idle time
Connection A
Connection B
Connection C
Server
handle data from A
handle data from A
Thread
handle data from B
Thread
handle data from C
Thread

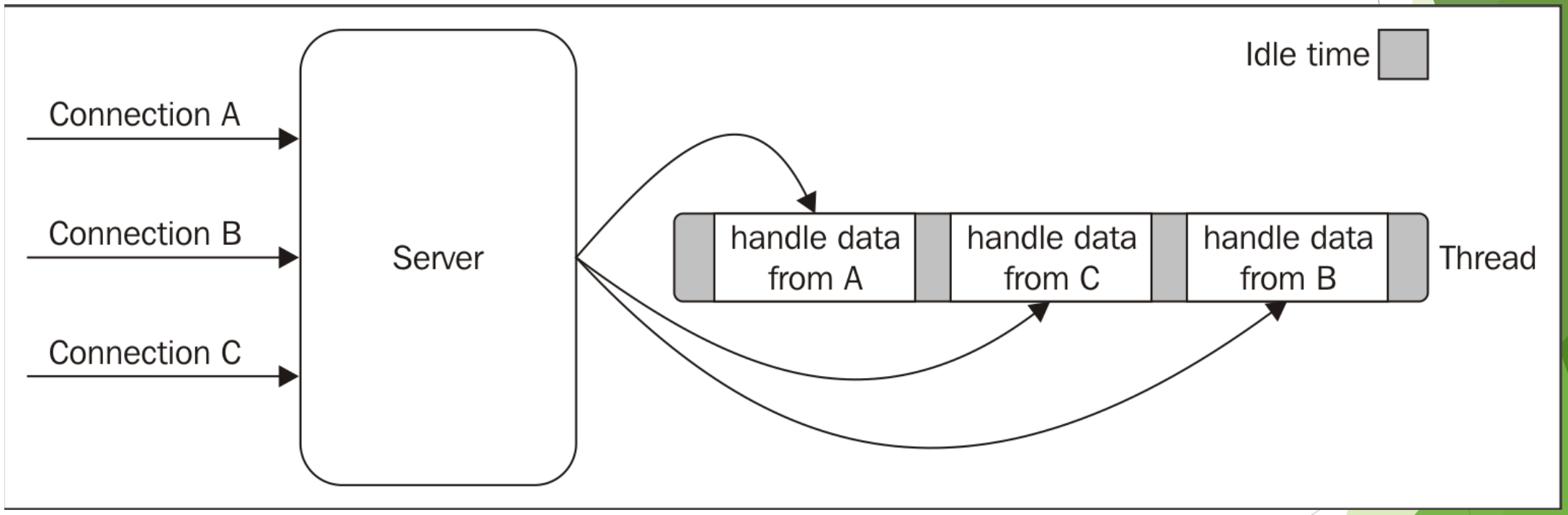
Non-blocking I/O: busy-waiting

```
resources = [socketA, socketB, pipeA];
while(!resources.isEmpty()) {
  for(i = 0; i < resources.length; i++) {
    resource = resources[i];
    //try to read
    let data = resource.read();
    if(data === NO_DATA_AVAILABLE)
      //there is no data to read at the moment
      continue;
    if(data === RESOURCE_CLOSED)
      //the resource was closed, remove it from the list
      resources.remove(i);
    else
      //some data was received, process it
      consumeData(data);
  }
}
```

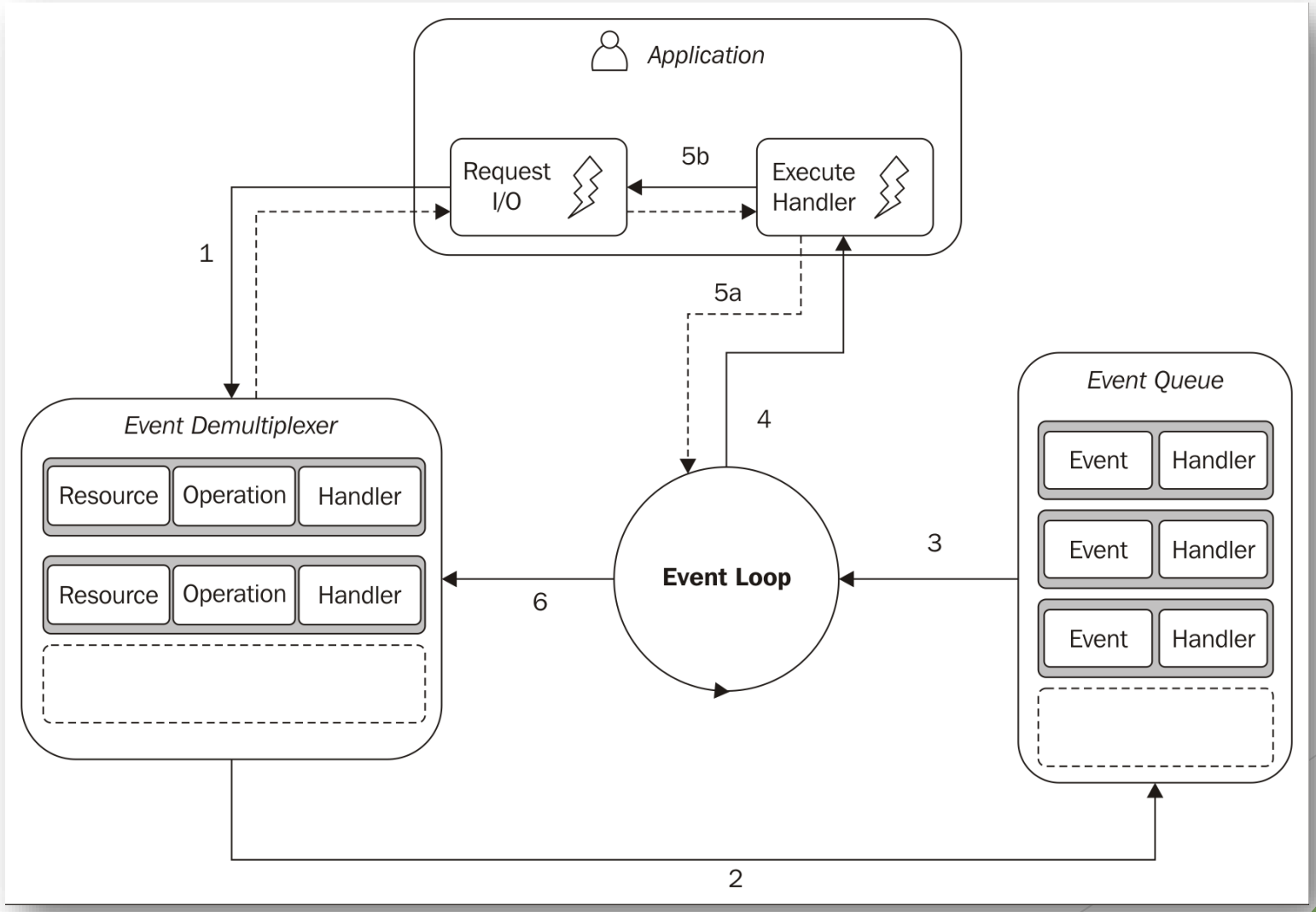
Non-blocking I/O: event demultiplexer

```
socketA, pipeB;
watchedList.add(socketA, FOR_READ);                          //[1]
watchedList.add(pipeB, FOR_READ);
while(events = demultiplexer.watch(watchedList)) {           //[2]
  //event loop
  foreach(event in events) {                                 //[3]
    //This read will never block and will always return data
    data = event.resource.read();
    if(data === RESOURCE_CLOSED)
      //the resource was closed, remove it from the watched list
      demultiplexer.unwatch(event.resource);
    else
      //some actual data was received, process it
      consumeData(data);
  }
}
```

Non-blocking I/O: single thread



Pattern (reactor) handles I/O by blocking until new events are available from a set of observed resources, and then reacts by dispatching each event to an associated handler.



Module

- Les modules sont les briques pour structurer des applications complexes
- L'un des problèmes majeurs avec JavaScript est l'absence d'espaces de noms
- Une technique populaire pour résoudre ce problème est le patron module révélateur (The revealing module pattern)
  - Définir une partie privée
  - exporter l'API publique
- CommonJS modules est la spécification de JS pour implémenter le patron module
- Node.js construis son système de module au dessus de cette spécification avec quelques extensions

CommonJS vs. ES Modules

Critère	CommonJS	ES Modules
Date de sortie	Vieux	Nouveau
Mots clés	require, exports	import, export
Chargement	Synchrone, par tout	Asynchrone, au début
Performance	Moins rapide	Plus rapide
Extension du fichier	js	mjs
Type dans package.json	commonjs	module

Définir un module CommonJS

- Tout est privés dans un module, il faut explicitement indiquer la partie publique avec la variable **module.exports**
- le contenu de cette variable est retourné quand le module défini est importé avec l'un des mot clés **require**

```
//load another dependency
const dependency = require('./anotherModule');

//a private function
function log() {
  console.log(`Well done ${dependency.username}`);
}

//the API to be exported for public use
module.exports.run = () => {
  log();
};
```

package.json and package-lock.json

package.json

- est un fichier de gestion des versions qui contient principalement la liste des dépendances (bibliothèques) dont votre projet node.js a besoin pour s'exécuter.
- Il inclut également d'autres méta-informations telles que les scripts, les informations sur l'auteur et la licence, la description, les propriétés du projet, etc.

package-lock.json

- est un fichier de verrouillage qui contient des informations sur les dépendances/paquets avec leurs numéros de version exacts qui ont été installés pour un projet node.js.

module.exports

- la variable **module.exports** peut contenir des valeurs, des objets ou encore des fonctions

```
module.exports.anExportedFunc = () => {};
module.exports.anExportedString = "this string is exported";

// or bundled together in an object
module.exports = {
  anExportedFunc,
  anExportedString,
};
```

require

- la fonction **require** permet de charger un module dans un autre
- Si le module n'est pas disponible localement, il faut l'installer avec npm
- elle charge seulement la valeur exportée **module.exports**
- les éléments de module.exports peuvent être chargés séparément

ship.js

```
...
module.exports = {
  containerA,
  containerB,
};
```

```
// importing the whole ship as a single variable
const ship = require("./ship.js");
console.log(ship.containerA);
console.log(ship.containerB);
// or directly importing containers through object destructuring
const { containerA, containerB } = require("./ship.js");
console.log(containerA);
console.log(containerB);
```

npm

- npm (Node Package Manager) est un outils permettant d'installer les package globalement ou dans un projet
- l'option –global ou –g indique que l'installation est global dans le dossier d'installation de node
- par défaut le package est installé dans un dossier appelé node_modules dans le répertoire courant
- npm sauvegarde le nom et la version du package installé dans le fichier package.json, la version exacte est sauvegardée dans package-lock.json
- pour sauvegarder le package en tant que dépendance de développement on ajoute l'option --save-dev
- le fichier package.json doit être crée manuellement

Node.js core modules

module	description
os	fournit des méthodes et des propriétés utilitaires liées au système d'exploitation
process	fournit des informations et un contrôle sur le processus Node.js actuel
fs	fournit une API pour interagir avec le système de fichiers
zlib	offre une fonctionnalité de compression
http	fournit une implémentation client/serveur HTTP

process

- permet l’accès aux paramètres passés par l’utilisateur dans la ligne de commande
- ces paramètres sont accessible via le tableau **argv**
- les variables d’environnement sont accessibles via l’objet **env**
- il est possible de fermer le processus node avec la fonction **exit()**

```
console.log(process.env)
{ SHELL: '/bin/bash’,
    USER: 'anna’,
    HOME: '/home/anna’,
    LOGNAME: 'anna', ...
}
```

```
node web.js testing app features
console.log(process.argv[2])
// Output: testing
```

fs

- offre quatre modes pour la manipulation des fichiers
  - synchrone
  - asynchrone:
    - callback
    - promises
    - stream

fs: synchrone

- les noms des fonctions synchrones se termine par Sync
  - existsSync(chemin): retourne vrai si le fichier ou le dossier existe
  - readFileSync(cheminFichier, options): retourne le contenu du fichier
  - writeFileSync(cheminFichier, options, données): écrit les données dans le fichier
  - copyFileSync(chemin1, chemin2)
  - renameSync(chemin1, chemin2)
  - unlinkSync(chemin): supprime le fichier
  - mkdirSync(chemin): ajoute un dossier
  - rmdirSync(chemin): supprime un dossier

exemple

```
// Include fs module
const fs = require('fs');

let data = "This is a file containing a collection of movies.";

// Writing to the file with the write file mode
fs.writeFileSync("movies.txt", data, {flag:"w"});
// Writing to the file 5 times with the append file mode
for (let i = 0; i < 5; i++) {
    fs.writeFileSync("movies.txt", "Movie " + i + "\n",{flag: "a+"});
  }
// Calling the readFileSync() method to read 'movies.txt' file
data = fs.readFileSync('./movies.txt',{encoding:'utf8', flag:'r'});

// Display the file data
console.log(data);
```

fs: asynchrone

- Par défaut, le module fs propose les fonctions asynchrones avec callback
  - ~~exists(chemin, callback)~~: retourne vrai si le fichier ou le dossier existe, **deprecated**
  - readFile(cheminFichier, options, callback): retourne le contenu du fichier
  - writeFile(cheminFichier, options, données, callback): écrit les données dans le fichier
  - copyFile(chemin1, chemin2, callback)
  - rename(chemin1, chemin2, callback)
  - unlink(chemin, callback): supprime le fichier
  - mkdir(chemin, callback): ajoute un dossier
  - rmdir(chemin, callback): supprime un dossier

exemple

```
let data = "This is a file containing a collection of movies.";

fs.writeFile("movies.txt", data,{flag: "w"},
  (err) => {
    if (err)
      console.log(err);
    else {
      console.log("File written successfully\n");
      console.log("The written has the following contents:");
      fs.readFile('movies.txt','utf8', (err, data) => {
        if (err)
        console.log(err);
      else
        console.log(data);
     });
    }
});
```

fs.promises: asynchrone avec Promises

- le sous module fs.promises propose les fonctions asynchrones avec Promises
- toutes les fonctions retournent des promesses qui contiennent éventuellement les résultats
  - access(cheminFichier, constants.R_OK): vérifie si le fichier existe et accessible, déclanche une erreur si ce n'est pas le cas
  - readFile(cheminFichier, options): retourne le contenu du fichier
  - writeFile(cheminFichier, options, données): écrit les données dans le fichier
  - copyFile(chemin1, chemin2)
  - rename(chemin1, chemin2)
  - unlink(chemin): supprime le fichier
  - mkdir(chemin): ajoute un dossier
  - rmdir(chemin): supprime un dossier

```
// Import the filesystem module
const fs = require('fs');
const fsPromises = fs.promises;
let data = "This is a file containing a collection of movies.";
async function main() {
    try {
        // Use the await keyword
        await fsPromises.writeFile("movies.txt", data);
        console.log("File written successfully");
        console.log("The written file has the following contents:");

        // Use the .then and .cathc methods
        fs.promises.readFile("./movies.txt")
            .then(function (result) {
                console.log("" + result);})
            .catch(function (error) {
                console.log(error);})

    } catch (err) {
        console.error(err);
    }
};
main();
```

stream: lecture asynchrone

- L'inconvénient des trois modes précédents est l'utilisation du buffer pour la lecture est l'écriture
- Le buffer présente les limites suivantes:
  - taille limitée
  - lecture discontinue
  - impossibilité d'utilisation des données avant le remplissage complet du buffer
- La solution est l'utilisation de readStream et WriteStream
- La lecture et l'écriture a partir des streams se fait avec les événements:
  - open: événement pour la lecture d'un morceau '**chunk**' de données reçu sur le flux
  - error: événement pour la gestion des erreurs
- Il est possible de lier directement un readStream et un writeStream avec la fonction pipe

exemple

```
// Use fs.createWriteStream() method to write the file
const writer = fs.createWriteStream('movies.txt', {flags: 'w'});
writer.write('This is a file containing a collection of movies.');

// Use fs.createReadStream() method to read the file
let reader = fs.createReadStream('movies.txt');

// Read and display the file data on console
reader.on('data', function (chunk) {
    console.log(chunk.toString());
});
reader.on('error', function (err) {
    console.log(err);
});
```

zLib

- permet la compression et lé décompression des fichiers en utilisant
- Buffer
  - gzipSync(buffer[,options]): retourne le buffer compressé
  - gzip(buffer[,options],callback): le buffer compressé est disponible dans le callback
  - unzipSynch (buffer[,options]): retourne le buffer décompressé
  - unzip (buffer[,options],callback): le buffer décompressé est disponible dans le callback
- Stream
  - createGzip: retourne un objet Gzip qui compresse les fichiers
  - createUnzip: retourne un objet Unzip qui décompresse les fichiers

exemple

```
const { createGzip } = require('zlib')
const { createWriteStream, createReadStream } = require('fs')

let readStream = createReadStream('movies.txt');
const gzip = createGzip();
const writeStream = createWriteStream(`movies.txt.gz`);
readStream
    .pipe(gzip)
    .pipe(writeStream)
    .on('finish', () => console.log(`File has been compressed: movies.txt`))
```

exemple 2

```
const { createGzip } = require('node:zlib');
const { pipeline } = require('node:stream');
const { createReadStream, createWriteStream } = require('node:fs');

const gzip = createGzip();
const source = createReadStream('movies.txt');
const destination = createWriteStream('movies.txt.gz');

pipeline(source, gzip, destination, (err) => {
  if (err) {
    console.error('An error occurred:', err);
    process.exitCode = 1;
  }
});
```

http

- permet l'implémentation d'un serveur http en node
- server = createServer(callback(req, res)): retourne un serveur et reçoit un callback qui sera appelé à la réception de chaque requête
  - req: représente la requête du client (http.IncomingMessage)
    - url
    - statusCode
  - res: représente la réponse (http.ServerResponse
    - write()
    - StatutCode
    - end()
- server.listen(port[, adresse]): écoute sur une adresse et un port

```
const http = require('http');
const fs = require('fs');
const port = (process.argv[2] || process.env.PORT || 3000);

const server = http.createServer((req, res) => {
  res.setHeader('Content-Type', 'text/html');
  switch (req.url) {
    case '/about':
      res.statusCode = 200;
      var data = fs.readFileSync('./pages/about.html');
      res.end(data);
      break;
    case '/about-us':
      res.writeHead(301, {
        location: "/about"
      });
      res.end();
      break;
    default:
      res.statusCode = 404;
      data = fs.readFileSync('./pages/404.html');
      res.end(data);
  }})
server.listen(port,'localhost');
console.log(`Server running at http://localhost:${port}/`);
```